Phytium飞腾

飞腾嵌入式 optee 的中断处理

（V1.1）

飞腾信息技术有限公司
www.phytium.com.cn
2023 年 12 月 7 日

版权所有 © 飞腾信息技术有限公司 2023。 保留一切权利。

未经本公司同意，任何单位、公司或个人不得擅自复制、翻译、摘抄本文档内容的部分或全部，不得以任何方式或途径进行传播和宣传。

商标声明

Phytium 和其他飞腾商标均为飞腾信息技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

特别提示

本文档仅作为使用指导，飞腾对本文档内容不做任何明示或暗示的声明或保证。本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

由于产品版本升级或其他原因，本文档内容会不定期进行更新，如有变更，恕不另行通知。

最新技术资源

飞腾嵌入式软件开源社区： https://gitee.com/phytium_embedded

或者访问飞腾软件开发者平台： https://service.phytium.com.cn/developer/

目录

1 中断概述 1
1.1 GIC 概述 1
1.2 中断类型 1
1.3 中断分组 2
2 异常向量表 3
2.1 概述 3
2.2 通用格式 3
2.3 OPTEE 的异常向量表 5
2.4 异常向量表实例 6
3 RPC 8
3.1 简介 8
3.2 RPC_FOREIGN_INTR 8
4 中断处理 10
4.1 概述 10
4.2 普通世界触发安全中断 10
4.3 安全世界触发非安全中断 11
A 术语和缩略语 15
B 更新记录 16

1 中断概述

1.1 GIC 概述

飞腾平台中，GIC 是通用中断控制器 Generic Interrupt Controller 的意思。随着硬件系统越来越复杂，多核成为系统的标准配置，那么中断怎么管理，怎么分配某个核来处理，成为一个非常重要的问题。

GIC 是系统中的一个重要组件，其作用是协调系统中的各种中断请求，在多任务和多设备的环境下，有效地管控中断信号的处理和响应。GIC 能够通过对中断的优先级和分级处理，使得系统可以高效地处理中断请求，减少设备间互相干扰，提高系统的稳定性和可靠性。

1.2 中断类型

本文档基于 ARMv8+GICv3 的硬件平台来描述，那么在这个平台下，所有中断被分为了四种类型，分别是：

1) SPI 中断（Shared Peripheral Interrupt）

共享外部中断类型，该类型中断不与特定的 Core 绑定，可以配置给任意 Core 或一组 Core 去处理。比如常见的硬件外部设备中断。

2) PPI 中断（Private Peripheral Interrupt）

私有外部中断类型，该类型中断是每个 Core 私有的，即一个特定的 PPI 中断只会配置到某一个特定的 Core 上。且其同一个中断号在每个 Core 上都可以是不同的 PPI 中断。

3) SGI 中断（Software Generated Interrupt）

软件通用中断类型，该类型中断并没有实际的物理连线，而是由软件通过写寄存器方式触发，它只支持边沿触发，通常用于 Core 间的通信。

4) LPI 中断（Locality-specific Peripheral Interrupt）

特定区域外围设备中断类型，该类型中断是一种基于消息的中断，外设不需要通过硬件中断线连接到 GIC 上，而可以通过向特定地址写入消息来触发中断。比如 PCIe 的 MSI 中断。

本文档后续说明中默认中断指 SPI 中断类型，其他三种中断类型不予考虑。

1.3 中断分组

基于 ARM Trustzone 技术，GICv3 进行了安全相关的扩展，在之前的中断类型基础上，对所有中断类型进行了分组，扩展了安全组 Secure Group。

在 GICv3 中，所有中断被分为以下三组：

1) Group 0（G0S）

默认在 EL3 环境中处理，如果 EL3 环境屏蔽中断处理时，可以通过配置在 Secure EL1 环境中处理。

2) Secure Group 1（G1S）

安全中断，默认在 Secure EL1 环境中处理，注意 ARMv8.4 以下没有 Secure EL2。

3) Non-secure Group 1（G1NS）

非安全中断，默认在 EL1 或 EL2 中处理。

通过分组的形式，某一个中断被分配到以上的某组之中，当该中断触发时，运行的 EL 环境不是该组处理的 EL 环境时，会通过触发 IRQ 或 FIQ 的方式，让系统切换到正确的 EL 环境下再进行处理。具体的触发方式如下图：

Table 4-3 Interrupt signals for two Security states when EL3 is using AArch64 state

Current Exception level	Group 0 interrupts	Group 1 interrupts	
		Secure	Non-secure
Secure EL1or EL0	FIQ	IRQ	FIQ
Non-secure EL1 or EL0	FIQ	FIQ	IRQ
Non-secure EL2	FIQ	FIQ	IRQ
EL3	FIQ	FIQ	FIQ

注意：上图是 ARM 官方文档《ARM Generic Interrupt Controller Architecture Specification(GIC architecture version 3.0 and version 4.0)》的 Table 4-3。

通过该图可以清楚得看到，除 EL3 之外，其他 EL 环境下，触发了本 EL 环境能处理的中断时，GIC 给出 IRQ 中断，否则是给出 FIQ 中断。所以，当安全世界出发非安全中断时，中断会被触发两次，第一次触发 FIQ 中断开启环境切换，第二次触发 IRQ 中断才是真正的处理过程。

本文档不考虑 Group 0 和 EL3 中断处理，因为在实际的飞腾平台中，EL3 是屏蔽中断的，EL3 只做消息的转发，不处理中断，这样做是为了减少系统运行在 EL3 的停留时间，以增加系统的安全性和健壮性。

2 异常向量表

2.1 概述

本文档描述的飞腾硬件平台具备 ARM Trustzone 硬件架构的能力，在该硬件架构下，系统被分为四个异常级别，分别是 EL0、EL1、EL2 和 EL3，其中 EL0 和 EL1 分别还具有安全世界的 EL 环境，分别是 S-EL0 和 S-EL1，ARM v8.4 以上版本才具有 S-EL2，本文档默认不考虑 S-EL2。

根据以上四个异常级别，系统一共有三个异常向量表，分别是 EL1、EL2 和 EL3 的异常向量表。这三个表的基地址分别存放在 VBAR_EL1、VBAR_EL2 和 VBAR_EL3 三个系统寄存器中。EL0 不需要异常向量表，EL0 是最低异常级别（用户级别），不能处理异常，所以 EL0 运行时发生的异常由 EL1 处理。

EL1 和 S-EL1 共享一个异常向量表，因为系统既不能同时处于 EL1 和 S-EL1 之中，也不能直接进行切换（需要通过 EL3 过渡）。当环境发生切换时，比如系统从 EL1 切换成 EL3，再切换成 S-EL1 时，对应的 VBAR_EL1 寄存器会在进入 S-EL1 时，重新指向一个 S-EL1 环境下软件系统的异常向量表。

举例说明，比如当 S-EL1 环境下软件系统是 OPTEE 时，新的异常向量表就是 OPTEE 中定义的一段符合异常向量表通用格式要求的代码段，具体是源码 thread_a64.S 中的 thread_excp_vect。而相关的 VBAR_EL1 寄存器值的 update 操作，会在每次 EL3 切换进入 OPTEE 时发生，具体是在初始化线程时调用 thread_init_vbar 函数完成。

注意：本文档只考虑 OPTEE（S-EL1 环境）中的异常向量表。其他普通世界的 EL1 异常向量表和 EL3 的异常向量表不是本文档的考虑范围，EL2 的异常向量表和虚拟化的实现有关，也不是本文档的考虑范围。

2.2 通用格式

飞腾系统平台规定的异常向量表具有通用的格式要求，具体如下图所示：

Table D1-7 Vector offsets from vector table base address

Exception taken from	Offset for exception type			
	Synchronous	IRQ or vIRQ	FIQ or vFIQ	SError or vSError
Current Exception level with SP_EL0.	0x000[a]	0x080	0x100	0x180
Current Exception level with SP_ELx, x>0.	0x200[a]	0x280	0x300	0x380
Lower Exception level, where the implemented level immediately lower than the target level is using AArch64.[b]	0x400[a]	0x480	0x500	0x580
Lower Exception level, where the implemented level immediately lower than the target level is using AArch32.[b]	0x600[a]	0x680	0x700	0x780

注意：上图是 ARM 官方文档《ARM Architecture Reference Manual（ARMv8, for ARMv8-A architecture profile）》的 Table D1-7。

通过该图中可以看出异常向量表通用格式的绝大部分信息，总结如下：

首先，一张表的大小是 2KB，16 进制表示是 0~0x800。一共有 16 个异常向量 Entry，每个异常向量占用的空间是 0x80，也就是 128 个字节。在 aarch64 体系下，一般指令是 32 位的，所以需要占用 4 字节，那么每个异常向量空间最多能存放 32 个指令。一般来说，比较复杂的处理，比如中断处理，都会通过一个跳转指令跳转到真正执行处理任务的代码段。

其次，在不考虑虚拟化的情况下，异常被分为了四种，分别是 Sync、IRQ、FIQ 和 SError，除了 Sync 是同步异常之外，其他三种都是异步异常。同步异常一般是指由指令主动发生的异常，比如指令运行时突然卡住，或者执行了某种特殊指令卡住，如果同步异常不处理，正常指令会一直卡住，不继续运行。同步异常主要有 abort 错误、SMC 指令、SVC 指令等。异步异常则是由外界触发的异常，正常指令会继续运行，只是置位相关标志提示处理，等待处理完成后需要清零标志。异步异常主要有中断、系统状态寄存器提示的某些系统错误等。

最后，根据异常发生时，系统正处于的环境，又分成了四种处理方式，四种异常对应四种处理方式组成 16 个异常向量。这四种异常发生时的环境分别是：

1) 当异常发生时，系统处于当前异常级别，并使用 SP_EL0 寄存器值作为 SP（堆栈指针）。

比如，当一个异常发生时，系统正处于 EL1 或 S-EL1，而该异常被送到 VBAR_EL1 指向的异常向量表中进行处理，而系统使用 SP_EL0 为 SP。

2) 当异常发生时，系统处于当前异常级别，并使用 SP_ELx 寄存器值作为 SP。

一般来说，在某个高异常级别下，可以访问两个 SP 寄存器，一个是 EL0 的 SP_EL0，一个是本级别的 SP_ELx，比如在 S-EL1 下，系统可以同时使用 SP_EL0 和 SP_EL1。

那么，这种环境是使用 SP_ELx 作为 SP，比如系统正处于 EL1 或 S-EL1 时，就是使用 SP_EL1 为 SP。

3) 当异常发生时，系统处于低一级的异常级别，且该级别运行的指令模式是 aarch64。

比如，当一个异常发生时，系统正处于 EL0 或 S-EL0，并以 aarch64 模式执行指令，而该异常被送到 VBAR_EL1 指向的异常向量表中进行处理，这种情况下，系统默认使用 SP_EL0 为 SP。

4) 当异常发生时，系统处于低一级的异常级别，且该级别运行的指令模式是 aarch32。

该环境和上一种环境非常类似，只是区别在于低一级的软件是以 aarch32 模式执行指令。

2.3 OPTEE 的异常向量表

OPTEE OS 运行在 S-EL1 的异常级别，所以 OPTEE 的异常向量表基地址是存放在 VBAR_EL1 寄存器中，负责 S-EL0 和 S-EL1 的异常处理。

OPTEE 一般运行在 aarch64 模式下，具体的异常向量表定义在 thread_a64.S 文件中，向量表名称是 thread_excp_vect。

OPTEE 的异常向量表按照通用格式也分为 16 个异常向量，其中有些异常向量的处理是不支持的，比如，SP_EL1 并不作为 SP，而是有其他作用，所以异常向量表中和 SP_EL1 有关的处理并不支持，都是死循环，另外对于 SError 也默认不支持，等待开发人员根据实际系统环境来添加处理流程，默认也是死循环。

本文档后续会重点介绍 OPTEE 异常向量表中的中断处理向量。

中断属于一种异步异常，系统定义了两种请求类型，分别是 IRQ（普通中断请求）和 FIQ（快速中断请求）。对应 OPTEE 异常向量表的实际情况，结合 TA 一般也运行在 aarch64 模式下，所以主要关注四个异常向量 Entry，分别是 el1_irq_sp0、el1_fiq_sp0、el0_irq_a64 和 el0_fiq_a64。这四个异常向量的处理最终又对应了 FIQ 和 IRQ 两个处理函数，分别是 foreign_intr_handler 和 native_intr_handler。

这里说明下：在 OPTEE 中，需要外部处理的中断称之为 foreign 中断（FIQ 类型触发），可以本地处理的中断称之为 native 中断（IRQ 类型触发）。

下表是基于 OPTEE v3.20 版本源码的异常向量表：

表 2.1 OPTEE 异常向量表

Offset	Entry	说明
0x000	el1_sync_sp0	在 S-EL1 下发生 Sync 异常，保存当前环境，进入 abort_handler 处理 abort 错误，然后恢复环境退出。
0x080	el1_irq_sp0	在 S-EL1 下发生 IRQ 中断，需要在本地处理。

0x100	el1_fiq_sp0	在 S-EL1 下发生 FIQ 中断，需要切换到其他异常级别进行处理。
0x180	el1_serror_sp0	死循环，不支持处理。
0x200	el1_sync_sp1	死循环，不支持处理。
0x280	el1_irq_sp1	
0x300	el1_fiq_sp1	
0x380	el1_serror_sp1	
0x400	el0_sync_a64	在 S-EL0 下（aarch64）发生 Sync 异常，根据 ESR 寄存器的 EC 值来判断是 svc 请求，还是 abort 错误，然后分别处理。
0x480	el0_irq_a64	在 S-EL0 下（aarch64）发生 IRQ 中断，需要在本地处理，这里“本地”是指异常向量表运行的本地，即 S-EL1。
0x500	el0_fiq_a64	在 S-EL0 下（aarch64）发生 FIQ 中断，需要切换到其他异常级别进行处理。
0x580	el0_serror_a64	死循环，不支持处理。
0x600	el0_sync_a32	在 S-EL0 下（aarch32）发生 Sync 异常，根据 ESR 寄存器的 EC 值来判断是 svc 请求，还是 abort 错误，然后分别处理。
0x680	el0_irq_a32	在 S-EL0 下（aarch32）发生 IRQ 中断，需要在本地处理，这里“本地”是指异常向量表运行的本地，即 S-EL1。
0x700	el0_fiq_a32	在 S-EL0 下（aarch32）发生 FIQ 中断，需要切换到其他异常级别进行处理。
0x780	el0_serror_a32	死循环，不支持处理。

2.4 异常向量表实例

对 OPTEE 的 bin 文件进行反汇编操作，可以看到异常向量表的实际情况，如下图：

```
00000000fc002000 <thread_excp_vect>:
thread_excp_vect():
/home/frankie/work/gitlab/fork/optee-4.0e/optee_os/core/arch/arm/kernel/thread_a64.S:288
    fc002000:   a90007e0    stp x0, x1, [sp]
    fc002004:   a9010fe2    stp x2, x3, [sp, #16]
/home/frankie/work/gitlab/fork/optee-4.0e/optee_os/core/arch/arm/kernel/thread_a64.S:289
    fc002008:   14000910    b   fc004448 <el1_sync_abort>
    ...

00000000fc002080 <el1_irq_sp0>:
el1_irq_sp0():
/home/frankie/work/gitlab/fork/optee-4.0e/optee_os/core/arch/arm/kernel/thread_a64.S:294
    fc002080:   a90007e0    stp x0, x1, [sp]
    fc002084:   a9010fe2    stp x2, x3, [sp, #16]
/home/frankie/work/gitlab/fork/optee-4.0e/optee_os/core/arch/arm/kernel/thread_a64.S:295
    fc002088:   1400097b    b   fc004674 <elx_irq>
    ...

00000000fc002100 <el1_fiq_sp0>:
el1_fiq_sp0():
/home/frankie/work/gitlab/fork/optee-4.0e/optee_os/core/arch/arm/kernel/thread_a64.S:300
    fc002100:   a90007e0    stp x0, x1, [sp]
    fc002104:   a9010fe2    stp x2, x3, [sp, #16]
/home/frankie/work/gitlab/fork/optee-4.0e/optee_os/core/arch/arm/kernel/thread_a64.S:301
    fc002108:   14000988    b   fc004728 <elx_fiq>
    ...
```

这是一个 thread_excp_vect 的实例，可以看出 el1_irq_sp0 和 el1_fiq_sp0 具体内容，都有两个 stp 指令，然后有个跳转 b 指令。在 OPTEE 临时使用的 reset_vect_table 实例中，可以看出最多一个表项可以有 32 条指令，如下图所示：

```
00000000fc001000 <reset_vect_table>:
SynchronousExceptionSP0():
/home/frankie/work/gitlab/fork/optee-4.0e/optee_os/core/arch/arm/kernel/entry_a64.S:689
    fc001000:   14000000    b   fc001000 <reset_vect_table>
    fc001004:   d503201f    nop
    fc001008:   d503201f    nop
    fc00100c:   d503201f    nop
    fc001010:   d503201f    nop
    fc001014:   d503201f    nop
    fc001018:   d503201f    nop
    fc00101c:   d503201f    nop
    fc001020:   d503201f    nop
    fc001024:   d503201f    nop
    fc001028:   d503201f    nop
    fc00102c:   d503201f    nop
    fc001030:   d503201f    nop
    fc001034:   d503201f    nop
    fc001038:   d503201f    nop
    fc00103c:   d503201f    nop
    fc001040:   d503201f    nop
    fc001044:   d503201f    nop
    fc001048:   d503201f    nop
    fc00104c:   d503201f    nop
    fc001050:   d503201f    nop
    fc001054:   d503201f    nop
    fc001058:   d503201f    nop
    fc00105c:   d503201f    nop
    fc001060:   d503201f    nop
    fc001064:   d503201f    nop
    fc001068:   d503201f    nop
    fc00106c:   d503201f    nop
    fc001070:   d503201f    nop
    fc001074:   d503201f    nop
    fc001078:   d503201f    nop
    fc00107c:   d503201f    nop
```

3 RPC

3.1 简介

RPC 是 Remote Procedure Call 的缩写，指远程过程调用，是一种通信机制，用于安全世界（Secure World）和普通世界（Normal World）之间进行函数调用和参数传递。

在 OPTEE 中，Secure World 内的程序可以通过 RPC 机制向 Normal World 中的程序发送请求，由后者去查询当前 Normal World 的各种状态，或者在 Normal World 中执行某些操作。

RPC 机制能够加强安全性，使 Secure World 和 Normal World 之间在数据保护和安全通信方面更加高效和安全。

3.2 RPC_FOREIGN_INTR

RPC 是一套完整的通信机制，一般应用程序之间使用的 RPC command 是一种应用类型，具体定义在 optee_rpc_cmd.h 中，通信的双方是 Secure World 的 TA 程序和 Normal World 的后台应用程序 tee-supplicant。

当 S-EL1 下触发 FIQ 中断时，运行的 foreign 中断处理程序需要通过 RPC 通知 Normal World 有需要处理的中断，使用的是 RPC_FOREIGN_INTR 类型的 RPC 消息，而这时候通信的双方是 Secure World 的 foreign 中断处理程序和 Normal World 的 Linux tee driver。

具体流程是：

1) foreign 中断处理程序主动封装 RPC 消息，并调用 SMC 指令。

SMC 指令执行会触发 EL3 的同步异常，由 EL3 异常向量表中对应的异常向量来处理。SMC 指令通过通用寄存器来输入输出参数。这里的第一个参数是 TEESMC_OPTEED_RETURN_CALL_DONE，第二个参数是 OPTEE_SMC_RETURN_RPC_FOREIGN_INTR。

2) EL3 runtime 服务转发 RPC 消息给 Normal World。

EL3 的同步异常处理，实际是转发消息给运行在 EL1 的 Linux tee driver。

3) EL1 的 tee driver 不做任何处理直接 return OK

以 Linux LTS 5.10.x 源码为例，具体源码在 drivers/tee/optee/rpc.c 文件中，在执行处理的函数 optee_handle_rpc 中有对 OPTEE_SMC_RPC_FUNC_FOREIGN_INTR 消息的处理过程，即不做任何处理。

4) EL1 下触发 IRQ 中断

由于 S-EL1 下触发的 FIQ 中断处理流程还没有完成，中断标志并未被清零，所以该中断一直处于未处理的状态。当系统异常级别切换到 EL1 时，这时候 GIC 判断中断需要以 IRQ 类型触发，因为该中断的类型是 G1NS，而当前异常级别是 EL1。这样就触发了 EL1 下的 IRQ 中断，然后进入 EL1 下的异常向量表进行正常 IRQ 的处理。

5) RPC 应答返回

Linux tee driver 负责发送 RPC 应答消息给回 OPTEE，在 OPTEE 系统下，对于 RPC 应答消息归类于一种 std_smc 处理，当 SMC 的第一个参数是 OPTEE_SMC_CALL_RETURN_FROM_RPC 时，这次 std_smc 处理的是一个 RPC 应答消息。

RPC 应答消息由 thread_resume_from_rpc 函数处理，主要是恢复 RPC 发送之前的 thread 环境，通过 thread 保存的上下文标志 threads[n].flags，可以判断这次是 RPC_FOREIGN_INTR 类型的 RPC 应答返回。

最后的处理函数是 thread_resume，还需要根据 SPSR 判断最终是返回 S-EL0 还是留在 S-EL1。

4 中断处理

4.1 概述

根据中断分组和飞腾平台的实际情况，系统运行在 Secure World 或 Normal World 时，都有可能产生 Secure Group 1（G1S）Non-Secure Group 1（G1NS）的中断。Group 0 的中断一般不予配置，EL3 也不进行中断处理，所以这里的 Secure World 不考虑 EL3 的情况。

当 Secure World 发生 G1S 的中断和 Normal World 发生 G1NS 的中断时，会触发 IRQ 普通中断，由于是中断正常的处理方式，会直接调用本地的 IRQ 中断处理函数完成对应的操作即可，比如 OPTEE 中的 native_intr_handler 处理当前 Secure World 中发生的 G1S 中断。

本章重点以 OPTEE 角度介绍两种需要中断路由的情况。一种是普通世界（Normal World）触发了安全中断（G1S），另一种是安全世界（Secure World）触发普通中断（G1NS）。

4.2 普通世界触发安全中断

在 Normal World 运行过程中，当系统触发了 G1S 中断，GIC 会以 FIQ 的中断类型进行触发。在飞腾平台下，Normal World 中的 FIQ 中断处理程序会知道这是一个需要 Secure World 中软件来处理的中断，就会通过 SMC 指令，请求 EL3 转发处理请求给 Secure World 进行处理。

以 OPTEE 为例，OPTEE OS 在初始化完成后，会提供一个向量表给 EL3 的软件，如下图：

```
FUNC thread_vector_table , : , .identity_map, , nobti
    b   vector_std_smc_entry
    b   vector_fast_smc_entry
    b   vector_cpu_on_entry
    b   vector_cpu_off_entry
    b   vector_cpu_resume_entry
    b   vector_cpu_suspend_entry
    b   vector_fiq_entry
    b   vector_system_off_entry
    b   vector_system_reset_entry
END_FUNC thread_vector_table
DECLARE_KEEP_PAGER thread_vector_table
```

可以看到，thread_vector_table 向量表中绿色的 Entry 是当前 OPTEE 已经实现的向量入口，而白色的 Entry 是占着位置但未实现内容的向量入口。

vector_fiq_entry 就是一个专用来处理 Normal World 发出的 FIQ 中断处理请求的 Entry。EL3 程序判断这次 SMC 指令是由于该情况导致的，就会直接调用 OPTEE 的 vector_fiq_entry 入口函数。

在 OPTEE 系统初始化的时候，vector_fiq_entry 是一定需要初始化好的，即便 OPTEE 不打算处理任何安全中断，也要把 vector_fiq_entry 初始化好，可以进行空操作，然后正确返回。这样设计的目的是防止系统有中断被意外设置成安全中断后，然后触发过来时没有处理，直接导致 CPU 运行卡死在这里。

vector_fiq_entry 主要是调用 OPTEE OS 的中断处理函数 itr_core_handler，该函数在 BSP 的主文件 main.c 中实现，可以由开发人员编写和修改。函数默认的处理流程是：

1) 读 IAR 寄存器得到该次触发中断的中断号 id。

2) 通过中断号 id 遍历中断处理函数链表，找到对应处理函数，并运行该函数。

OPTEE 中定义了一个全局的链表 handlers 用于保存安全中断的各种处理函数和相关信息，这样也方便了安全中断在系统中的注册。通过这个设计，OPTEE 的中断注册，实际就是增加一个链表项信息到全局链表 handlers 中即完成了注册过程。可以通过 itr_alloc_add 函数，将中断号和中断处理函数，以及和该中断处理相关的信息填入到 handlers 的一个新链表项中。

如果一个安全中断处理过程中，没有从 handlers 中找到对应的中断处理函数，则说明该安全中断并没有在 OPTEE 中注册。这时候，处理流程会调用 disable 接口将该安全中断进行屏蔽，只有当安全中断被正确注册时，系统才会调用 enable 接口打开屏蔽位。

3) 将 id 值写回 EOIR 寄存器，中断处理结束，并返回 Normal World。

这里 EOIR（End Of Interrupt Register）寄存器的写操作表示了该安全中断处理已经完成。接下来，vector_fiq_entry 需要结束调用，通过执行参数是 TEESMC_OPTEED_RETURN_FIQ_DONE 的 SMC 指令，回到 EL3 状态下，并通过参数提示 Normal World 中的相关程序，本次 FIQ 已经处理完毕。

4.3 安全世界触发非安全中断

在 Secure World 运行过程中，当系统触发了 G1NS 中断，GIC 也会以 FIQ 的中断类型进行触发，但是相比 Normal World 触发 G1S 中断而言，处理流程会更加复杂一点，处理流程如下图所示：

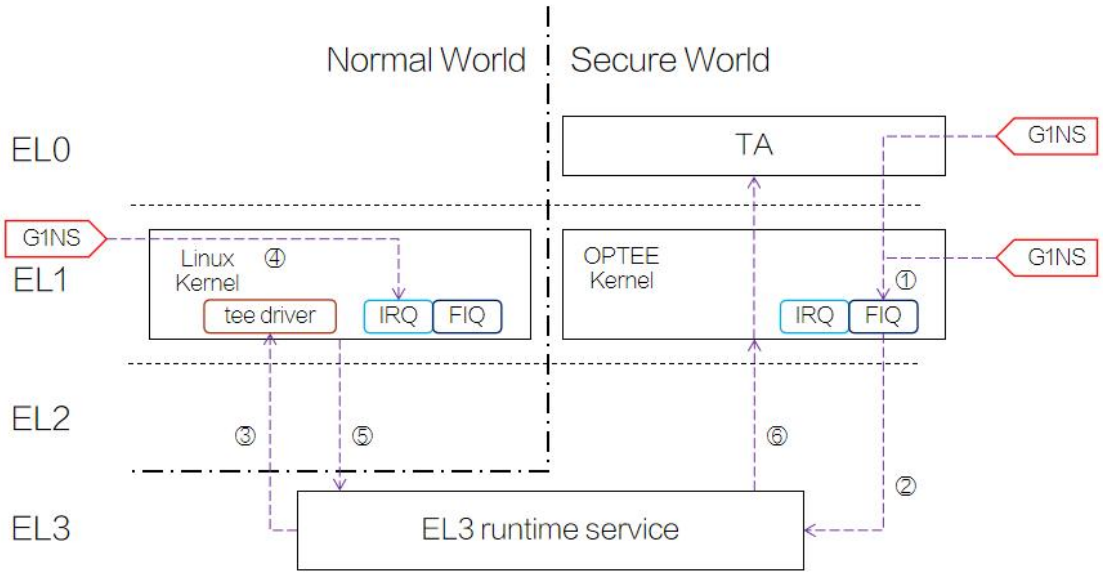

图 4.1 安全世界触发非安全中断的处理流程图

第①步：当 Secure World 触发 G1NS 中断时，有可能程序是运行在 S-EL0 的 TA 程序中，也有可能运行在 OPTEE Kernel 之中。这两种情况对应 OPTEE 两个不同的异常向量入口，但最终都是触发 OPTEE 在 S-EL1 下运行 foreign_intr_handler 函数。

foreign_intr_handler 函数主要处理操作是：

1) 把当前 thread 运行环境进行保存操作。

2) 调用 thread_state_suspend 将当前 thread 挂起来。

3) 调用 thread_foreign_intr_exit，将 RPC_FOREIGN_INTR 类型的 RPC 消息通过 SMC 指令发给 EL3 进行处理。

第②步：当 SMC 指令执行时，系统主动产生一个同步异常，由于是 S-EL1 状态下发生的同步异常，需要有更高级的异常级别进行处理，这样系统进入 EL3 状态下的同步异常向量入口进行处理。

第③步：EL3 runtime service 提供 EL3 状态下的程序运行，主要负责两个世界的消息转发和世界切换的操作。在完成一些世界切换必须的操作后，EL3 程序会转发消息给 EL1 状态下的 Linux Kernel 的入口程序，最后消息会转发到 Linux tee driver 模块中进行处理。

第④步：当系统切换到 EL1 状态下运行时，由于 G1NS 中断在 Secure World 下触发的是 FIQ 中断，而且并没有处理结束，所以这时候会马上触发 EL1 下的 IRQ 中断。通过 EL1 下 Linux 系统的 IRQ 中断处理流程，将本次中断事件正常处理完成。

当 IRQ 中断处理流程完成后，中断响应处理就实际已经完成了，但中断真正发生的地方是在 Secure

World 下，这时候系统需要回到中断真正发生的位置继续运行程序。

第⑤步：EL1 下的 IRQ 中断处理完成之后，Linux 运行恢复到 tee driver 模块中，这时候，tee driver 并不需要处理任何事情，直接使用 RPC 应答消息，返回 OK 即可。

封装相应的消息，调用 SMC 指令，产生同步异常，再次进入到 EL3 下同步异常向量入口进行处理。

第⑥步：RPC 应答消息属于 OPTEE 中的 std smc 消息，从 vector_std_smc_entry 向量入口进入 OPTEE Kernel 运行。系统能判断是 RPC_FOREIGN_INTR 类型的 RPC 应答返回，会根据流程调用 thread_resume_from_rpc 函数将之前挂起的 thread 进行恢复运行，并将之前保存的程序运行地址和函数参数等信息恢复到 SP 寄存器中。如果之前的中断点是 S-EL0 下的，还要进行 S-EL1 到 S-EL0 的环境切换。

A 术语和缩略语

名称	全称	解释
OPTEE	Open Portable TEE	一个开源的、可信执行环境运行软件实现的项目，包括操作系统、运行在非安全执行环境中的交互驱动、支持程序和库。
TEE	Trusted Execution Environment	可信的执行环境，在 ARM 的安全架构中，属于安全世界的执行环境。
REE	Rich Execution Environment	丰富的执行环境，在 ARM 的安全架构中，属于非安全世界的执行环境。
TA	Trusted Application	运行在安全世界的应用程序。
CA	Client Application	运行在非安全世界的应用程序，负责和安全世界的应用程序进行交互。

B 更新记录

更新日期	版本号	编制人	说明
2023-08-25	1.0	邓强	首次发布。
2023-12-7	1.1	邓强	增加异常向量表的实例说明；修订几个文档说明，以使表述更加严谨。